

P2P - II

Lecture 5

January 19, 2026

Send/Recv Example

```
int count = atoi (argv[1]);  
if (myrank == 0)  
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);  
else  
    if (myrank == 1)  
        MPI_Recv (buffer, count, MPI_CHAR, 0, 1, MPI_COMM_WORLD, &status);
```

`mpirun -np 2 ./exe 10`

Will this run/block/give compile-time error?

Send/Recv Example

```
int count = atoi (argv[1]);  
if (myrank == 0)  
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);  
else  
    if (myrank == 1)  
        MPI_Recv (buffer, count, MPI_CHAR, 0, 1, MPI_COMM_WORLD, &status);
```

```
Fatal error in MPI_Recv: Message truncated, error stack:  
MPI_Recv(200).....: MPI_Recv(buf=0x7ffd3c917b30, count=10, MPI_CHAR,  
src=0, tag=1, MPI_COMM_WORLD, status=0x7ffd3c917c40) failed  
MPIDI_CH3U_Receive_data_found(131): Message from rank 0 and tag 1 truncated; 40 bytes  
received but buffer size is 10
```

Send/Recv Example

```
if (myrank == 0)
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);
else
    if (myrank == 1)
        MPI_Recv (buffer, countchar, MPI_CHAR, 0, 1, MPI_COMM_WORLD,
&status);
```

No runtime error, programmer's responsibility

Example – Contd.

```
int count = atoi (argv[1]), recvcount = 0;
```

```
if (myrank == 0)
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);
else
    if (myrank == 1) {
        MPI_Recv (buffer, count_, MPI_CHAR, 0, 1, MPI_COMM_WORLD, &status);
        MPI_Get_count (&status, MPI_CHAR, &recvcount);
    }

printf ("%d %d %d\n", myrank, count, recvcount);
```

```
class > mpirun -np 2 ./datatype 10
0 10 0
1 10 40
```

Example – Contd.

```
int count = atoi (argv[1]), recvcount = 0;
```

```
if (myrank == 0)
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);
else
    if (myrank == 1) {
        MPI_Recv (buffer, count_, MPI_CHAR, 0, 1, MPI_COMM_WORLD, &status);
        MPI_Get_count (&status, MPI_CHAR, &recvcount);
    }

printf ("%d %d %d\n", myrank, count, recvcount);
```

```
class > mpirun -np 4 ./datatype 10
2 10 0
3 10 0
0 10 0
1 10 40
```

```

int main( int argc, char *argv[])
{
    int myrank, size;
    double sTime, eTime, time;
    MPI_Status status;

    MPI_Init(&argc, &argv);

    int count = atoi (argv[1]);
    int buf[count];

    MPI_Comm_rank(MPI_COMM_WORLD, &myrank) ;
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // initialize data
    for (int i=0; i<count; i++)
        buf[i] = myrank+i;

    sTime = MPI_Wtime();
    if (myrank == 0)
        MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);
    eTime = MPI_Wtime();
    time = eTime - sTime;

    printf ("%lf\n", time);

    MPI_Finalize();
    return 0;
}

```

Incorrect code

There is a bug.

0.000000

0.000002

Programming bug, no runtime
Or compile-time error!

Matching MPI_Recv Not Posted

Unsafe code

```
int main( int argc, char *argv[])
{
    int myrank, size;
    MPI_Status status;
    double sTime, eTime, time;

    MPI_Init(&argc, &argv);

    int count = atoi (argv[1]);
    int buf[count];

    MPI_Comm_rank(MPI_COMM_WORLD, &myrank) ;
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // initialize data
    for (int i=0; i<count; i++)
        buf[i] = myrank+i;

    sTime = MPI_Wtime();
    if (myrank == 0)
        MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);
    eTime = MPI_Wtime();
    time = eTime - sTime;

    printf ("%lf\n", time);

    MPI_Finalize();
    return 0;
}
```

norecv.c

```
class $ mpirun -np 2 ./norecv 10
0.000007
0.000001
class $ mpirun -np 2 ./norecv 100
0.000015
0.000002
class $ mpirun -np 2 ./norecv 1000
0.000036
0.000001
class $ mpirun -np 2 ./norecv 10000
0.000002
0.000024
class $ mpirun -np 2 ./norecv 100000
0.000001
[mpiexec@csews7] Sending Ctrl-C to processes as requested
[mpiexec@csews7] Press Ctrl-C again to force abort
class $
```


Send/Recv Example

```
int buf[100], count=10;  
if (myrank == 0)  
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);  
else  
    if (myrank == 1)  
        MPI_Recv (&buf[10], count, MPI_INT, 0, 1, MPI_COMM_WORLD,  
        &status);
```

MPI_Recv Example

```
// initialize data
for (int i=0; i<count; i++)
    buf[i] = myrank+i;

printf ("%d: Before send: %d %d %d\n", myrank, buf[0], buf[1], buf[2]);

if (myrank == 0)
    MPI_Send (buf, 2, MPI_INT, 1, 1, MPI_COMM_WORLD);
else
    MPI_Recv (buf, 2, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);

printf ("%d: After send: %d %d %d\n", myrank, buf[0], buf[1], buf[2]);
```

```
mpirun -np 2 ./a.out 100
0: Before send: 0 1 2
0: After send: 0 1 2
1: Before send: 1 2 3
1: After send: 0 1 3
```

If send message size is smaller than the receive count, then only those locations corresponding to the (shorter) message are modified.

Multiple Send/Recv



Unsafe code

```
if (myrank == 0) {  
    buf[0] = 24;  
    MPI_Send (buf, 1, MPI_INT, 1, 1, MPI_COMM_WORLD);  
    buf[0] = 25;  
    MPI_Send (buf, 1, MPI_INT, 1, 2, MPI_COMM_WORLD);  
}  
  
else  
    if (myrank == 1)  
        MPI_Recv (buf, 1, MPI_INT, 0, 2, MPI_COMM_WORLD, &status);  
  
printf ("%d %d\n", myrank, buf[0]);
```

Semantics of P2P

```
if (myrank == 0) {  
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD),  
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);  
}  
else  
    if (myrank == 1) {  
        MPI_Recv (buf, count, MPI_INT, 0, 1, MPI_COMM_WORLD, &status),  
        MPI_Recv (buf, count, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);  
}  
printf ("%d %d\n", myrank, count);
```

Messages are nonovertaking

Semantics of P2P

Order. Messages are nonovertaking:

If a sender sends two messages in succession to the same destination, and both match the same receive, then this operation cannot receive the second message if the first one is still pending.

If a receiver posts two receives in succession, and both match the same message, then the second receive operation cannot be satisfied by this message, if the first one is still pending.

-- MPI Report

Multiple Send/Recv

0	24	25
1	25	24

```
if (myrank == 0) {  
    buf[0] = 24;  
    MPI_Send (&buf[0], 1, MPI_INT, 1, 1, MPI_COMM_WORLD);  
    buf[1] = 25;  
    MPI_Send (&buf[1], 1, MPI_INT, 1, 2, MPI_COMM_WORLD);  
}  
  
else  
  
    if (myrank == 1) {  
        MPI_Recv (&buf[1], 1, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);  
        MPI_Recv (&buf[0], 1, MPI_INT, 0, 2, MPI_COMM_WORLD, &status);  
    }  
  
printf ("%d %d %d\n", myrank, buf[0], buf[1]);
```

Timing Send/Recv (timingSend.c)

```
MPI_Comm_rank(MPI_COMM_WORLD, &myrank) ;
MPI_Comm_size(MPI_COMM_WORLD, &size);

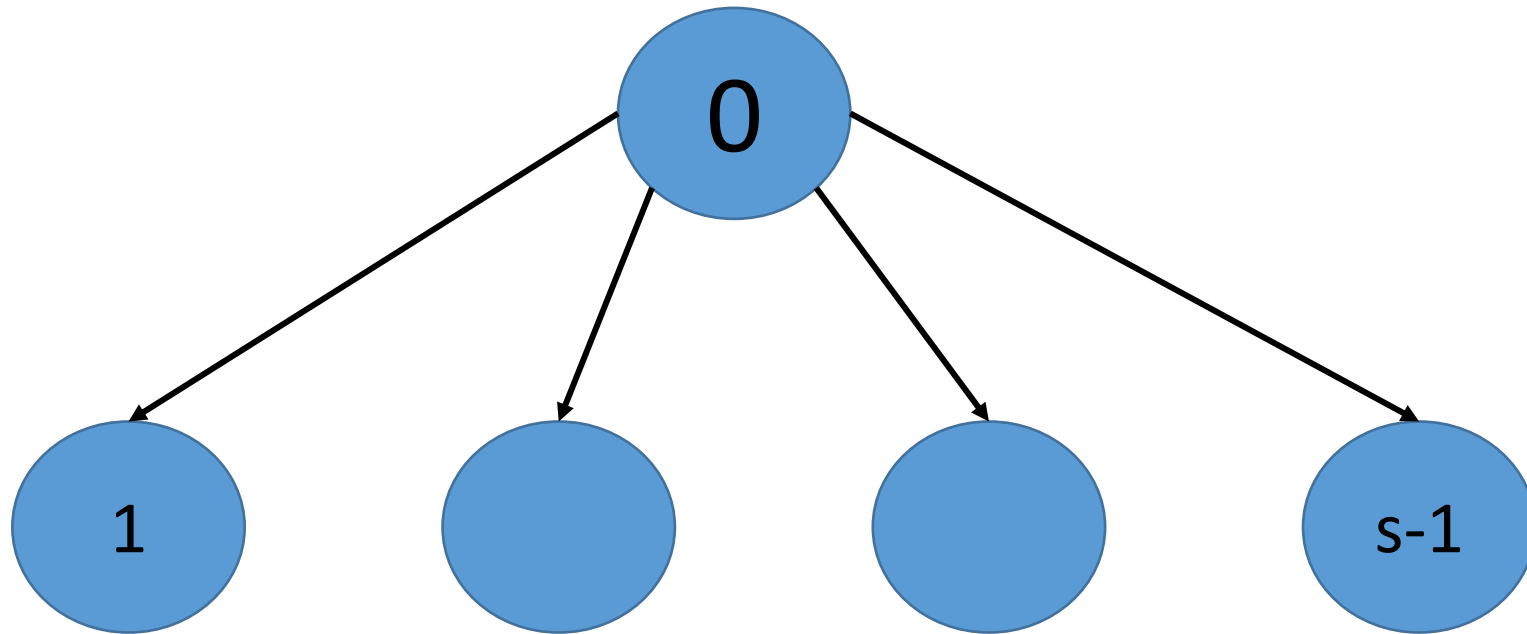
// initialize data
for (int i=0; i<count; i++)
    buf[i] = myrank+i;

sTime = MPI_Wtime();
if (myrank == 0)
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);
else
    if (myrank == 1)
        MPI_Recv (buf, count, MPI_INT, 0, 1, MPI_COMM_WORLD, &status);
eTime = MPI_Wtime();
time = eTime - sTime;
```

Timing Variation

```
class mpirun -np 2 ./timingSend 100
0.000002
0.000039
class mpirun -np 2 ./timingSend 100
0.000002
0.000009
class mpirun -np 2 ./timingSend 10000
0.000022
0.000039
class mpirun -np 2 ./timingSend 10000
0.000022
0.000030
class mpirun -np 2 ./timingSend 1000000
0.000426
0.000477
class mpirun -np 2 ./timingSend 1000000
0.000398
0.000403
class mpirun -np 2 ./timingSend 100000000
0.052907
0.052171
class mpirun -np 2 ./timingSend 100000000
0.055681
0.056406
```


One-to-many Sends Recap



One-to-many Sends (Naïve Algorithm) Recap

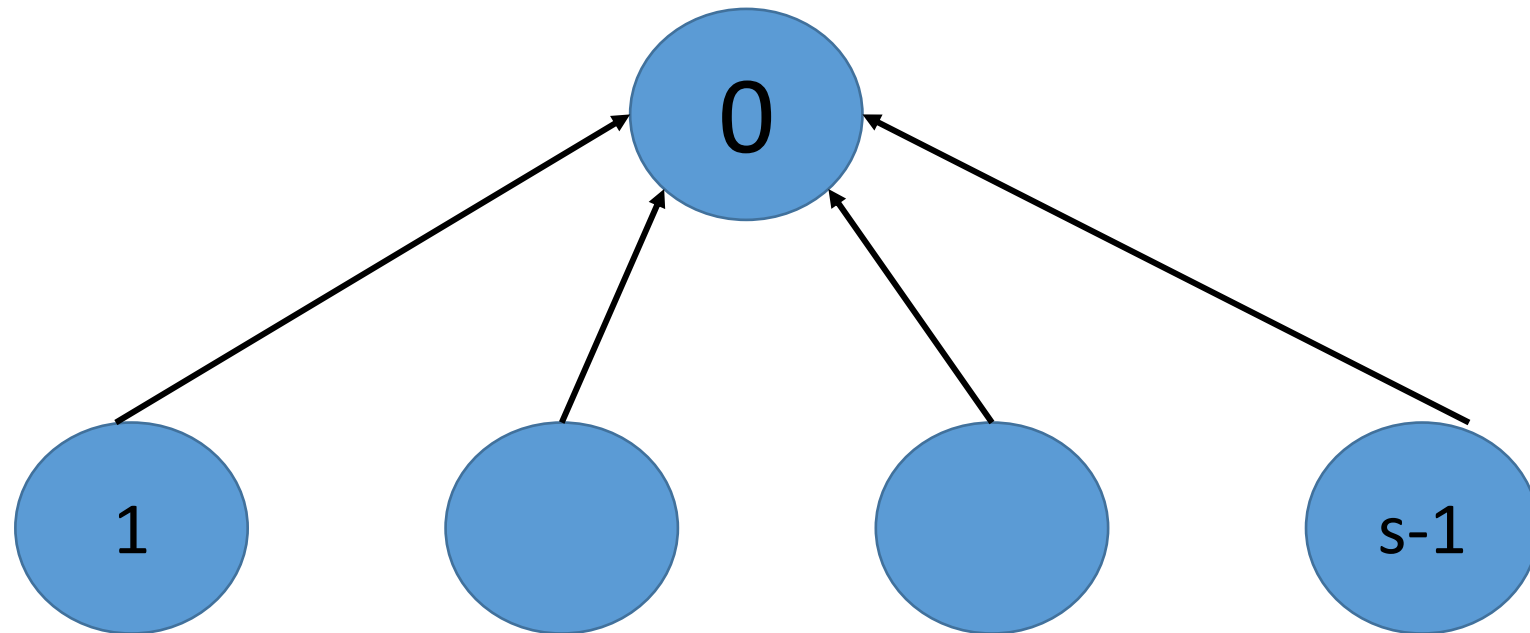
```
sTime = MPI_Wtime();
if (myrank == 0)
    for (int r=1; r<size; r++)
        MPI_Send (buf, count, MPI_INT, r, r, MPI_COMM_WORLD);
else
    MPI_Recv (buf, count, MPI_INT, 0, myrank, MPI_COMM_WORLD, &status);
eTime = MPI_Wtime();
time = eTime - sTime;

// obtain max time
MPI_Reduce (&time, &maxTime, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);
if (!myrank) printf ("%lf\n", maxTime);
```

One-to-many Sends Performance Analysis

```
class $ for i in `seq 1 5`; do mpirun -np 8 ./onetomanyseeds 1000000 ; done
0.004122
0.005187
0.004641
0.004474
0.004451
class $ for i in `seq 1 5`; do mpirun -np 8 -hosts csews1:2,csews10:2,csews20:2,csews30:2 ./onetomanyseeds 1000000 ; done
0.205539
0.215736
0.217059
0.214883
0.208585
class $
```

Many-to-one Sends



Many-to-one Sends

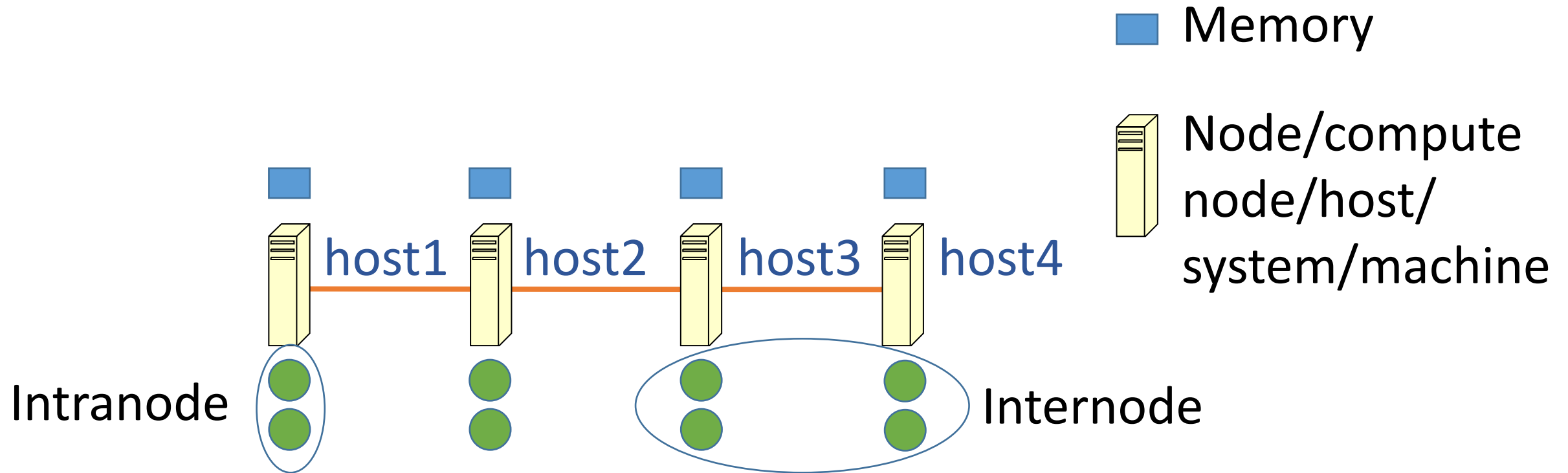
Receive
in order

```
if (myrank)
{
    MPI_Send(arr, 20, MPI_INT, 0, myrank, MPI_COMM_WORLD);
}
else
{
    int count, recvarr[size][20];
    for (int i=1; i<size; i++)
    {
        MPI_Recv(recvarr[i], 20, MPI_INT, i, i, MPI_COMM_WORLD, &status); // slower
        printf("Rank %d of %d received from rank %d with tag %d\n", myrank, size, status.MPI_SOURCE, status.MPI_TAG);
    }
}
```

MPI_Send (Blocking send)

MPI_Recv (Blocking receive)

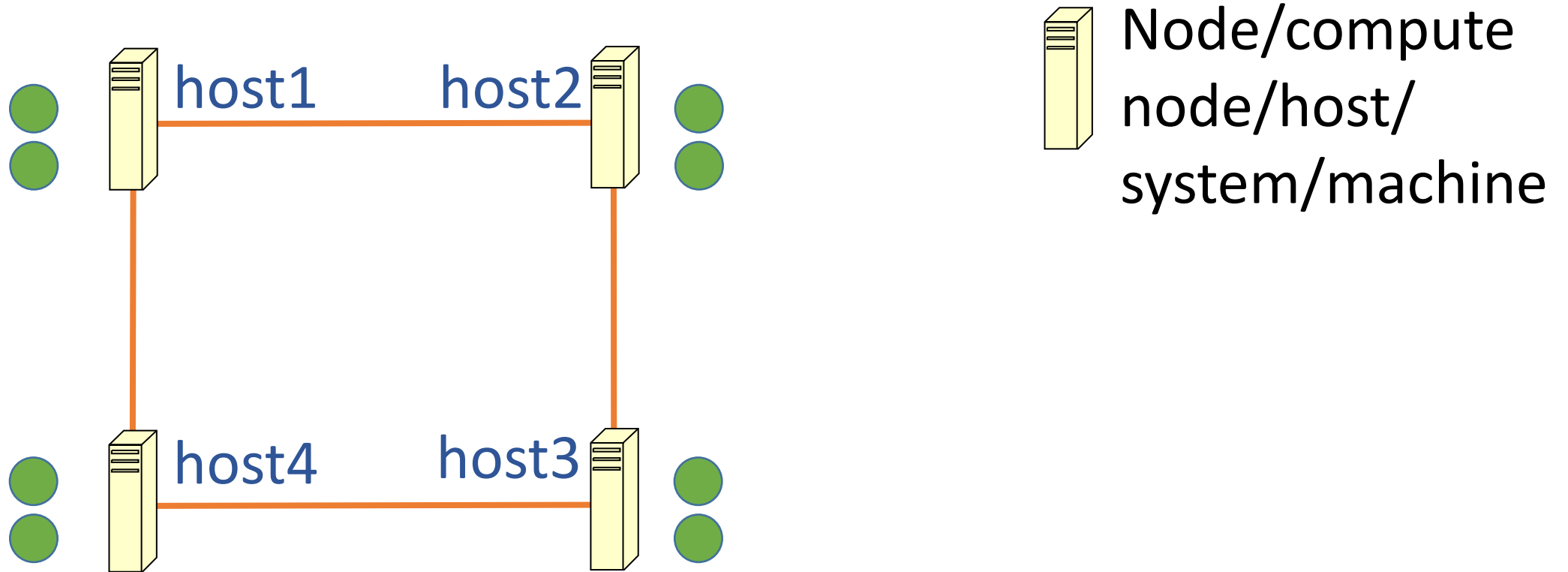
MPI Program Execution Order



```
mpixec -n 8 -hosts host1,host2,host3,host4 ./exe
```

```
mpixec -n 8 -hosts host1:2,host2:2,host3:2,host4:2 ./exe
```

MPI Program Execution Order



MPI_ANY_*

int MPI_Recv (void *buf, int count, MPI_Datatype datatype, int source, int tag, MPI_Comm comm, MPI_Status *status)

- MPI_ANY_SOURCE
 - Receiver may specify wildcard value for source
- MPI_ANY_TAG
 - Receiver may specify wildcard value for tag

Receive Out-of-order

a send operation
must specify a unique
receiver

```
if (myrank)
{
    MPI_Send(arr, 20, MPI_INT, 0, myrank, MPI_COMM_WORLD);
}
else
{
    int count, recvarr[size][20];
    for (int i=1; i<size; i++)
    {
        //MPI_Recv(recvarr[i], 20, MPI_INT, i, i, MPI_COMM_WORLD, &status); // may be slower
        MPI_Recv(recvarr[i], 20, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
        printf("Rank %d of %d received from rank %d with tag %d\n", myrank, size, status.MPI_SOURCE, status.MPI_TAG);
    }
}
```

H.W.: Rectify the
receive buffer location

No specific
order

Query MPI_Status [version 4.0 onwards]

`MPI_Status_get_source(status, source)`

IN status status from which to retrieve source rank (status)

OUT source rank set in the `MPI_SOURCE` field (integer)

`MPI_Status_get_tag(status, tag)`

IN status status from which to retrieve tag (status)

OUT tag tag set in the `MPI_TAG` field (integer)

Receive Out-of-order

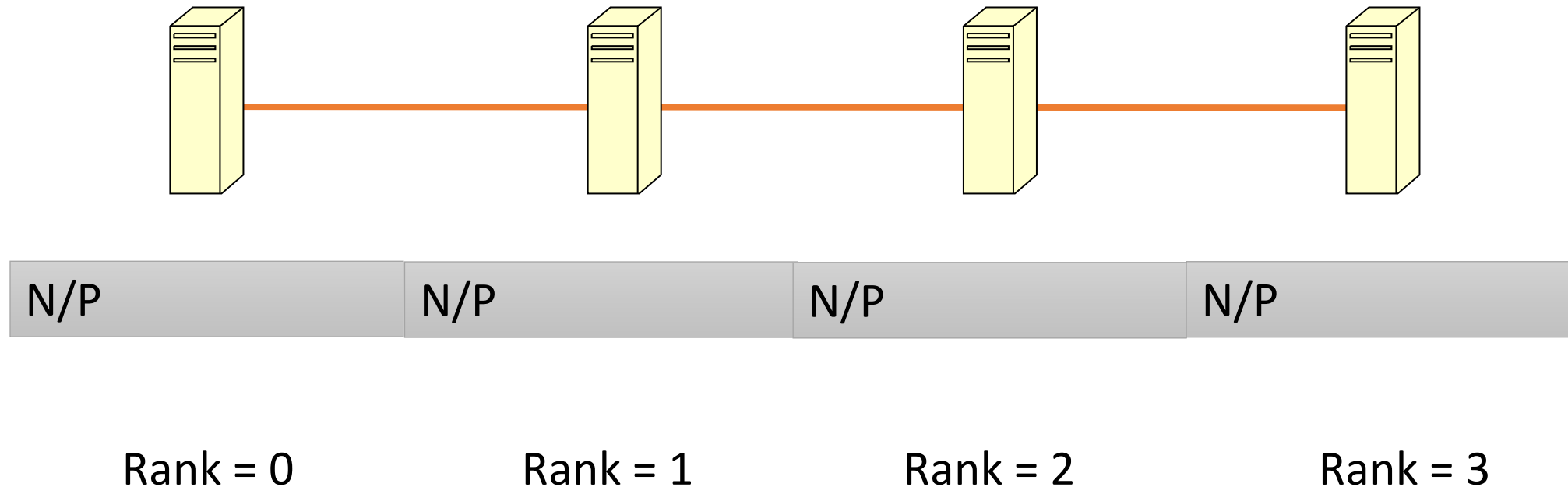
```
if (myrank)
{
    MPI_Send(arr, 20, MPI_INT, 0, myrank, MPI_COMM_WORLD);
}
else
{
    int count, recvarr[size][20];
    for (int i=1; i<size; i++)
    {
        //MPI_Recv(recvarr[i], 20, MPI_INT, i, i, MPI_COMM_WORLD, &status); // may be slower
        MPI_Recv(recvarr[i], 20, MPI_INT, MPI_ANY_SOURCE, MPI_ANY_TAG, MPI_COMM_WORLD, &status);
        printf("Rank %d of %d received from rank %d with tag %d\n", myrank, size, status.MPI_SOURCE, status.MPI_TAG);
    }
}
```

- This may result in similar performance if
- MPI implementation performs buffering at the receiver side
 - Fewer process counts/node counts

- Design MPI implementation-agnostic code
- Design network-agnostic code
- Design configuration-agnostic code

Receive Sub-array at 0

- Array a is present in all ranks
- Each rank computes portion of a



Receive Sub-array at Rank 0

```
stime = MPI_Wtime();
if (rank)
{
    MPI_Send(a+sidx, N/numtasks, MPI_DOUBLE, 0, rank, MPI_COMM_WORLD);
}
else
{
    int recvarr[numtasks][N/numtasks];
    for (int r=1; r<numtasks; r++)
    {
        MPI_Recv(recvarr[r], N/numtasks, MPI_DOUBLE, r, r, MPI_COMM_WORLD, &status);
    }
}
etime = MPI_Wtime();
cotime = etime - stime;
```

communication time

Sum of Squares of N numbers

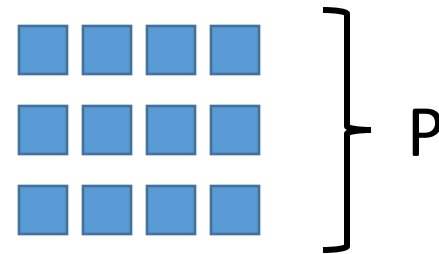
Serial

```
for i = 1 to N  
  sum += a[i] * a[i]
```



Parallel

```
for i = 1 to N/P  
  sum += a[i] * a[i]  
collate result
```



Sum of Squares

Fix the start and end positions



```
for i = 1 to N/P  
sum += a[i] * a[i]
```

Rank = 0

```
for i = 1 to N/P  
sum += a[i] * a[i]
```

Rank = 1

```
for i = 1 to N/P  
sum += a[i] * a[i]
```

Rank = 2

```
for i = 1 to N/P  
sum += a[i] * a[i]
```

Rank = 3

```
// Assume the large array (size N) is present in all ranks (P)
```

```
for (?: ?; ?)
```

```
    localsum += a[i] * a[i]
```

```
// Collect localsum, add up at one of the ranks
```

```
if (!rank)
```

```
    recv from all ranks (...)
```

```
else
```

```
    send (0)
```



```
// Assume the large array (size N) is present in all ranks (P)
```

```
for (int i = N/P * rank ; i < N/P * (rank+1) ; i++)  
    localsum += a[i] * a[i];
```

```
// Collect localsum, add up at one of the ranks
```

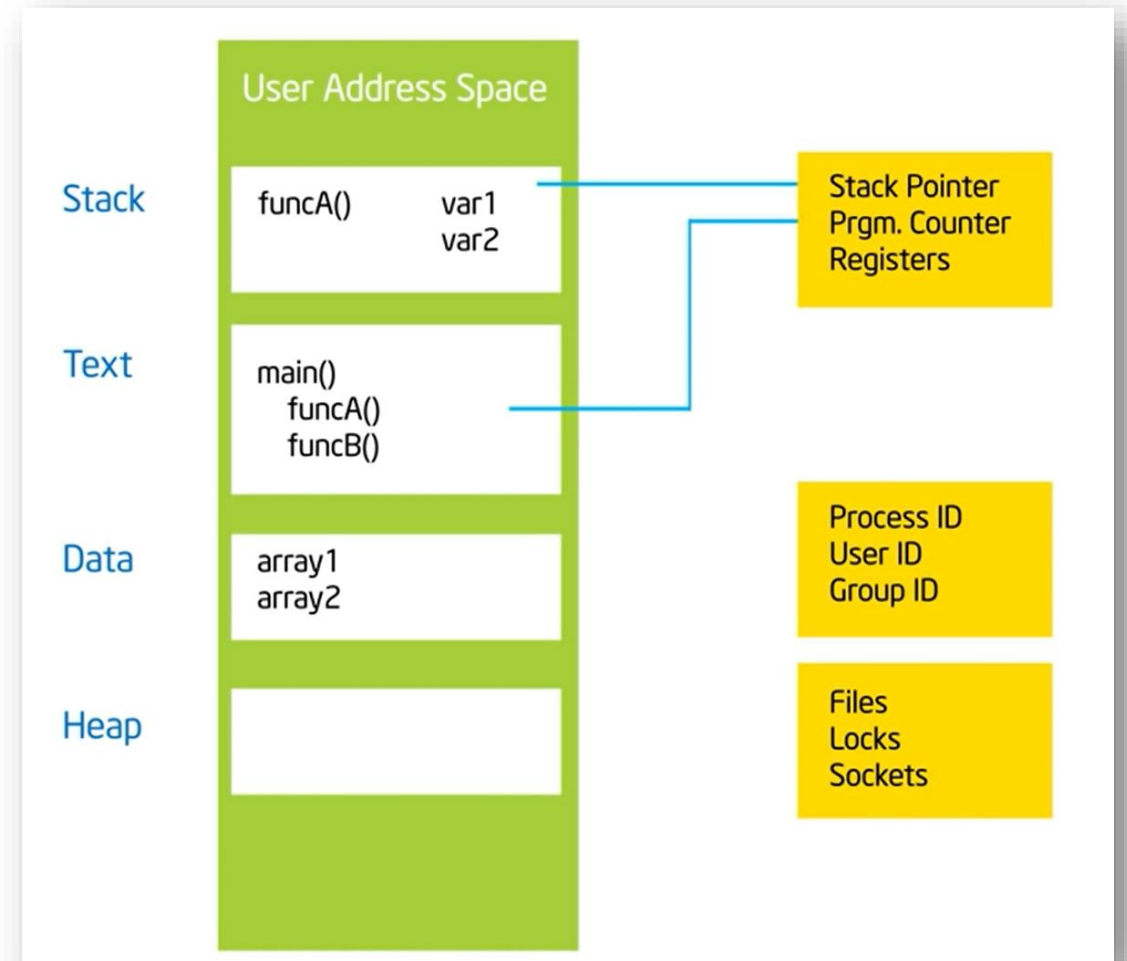
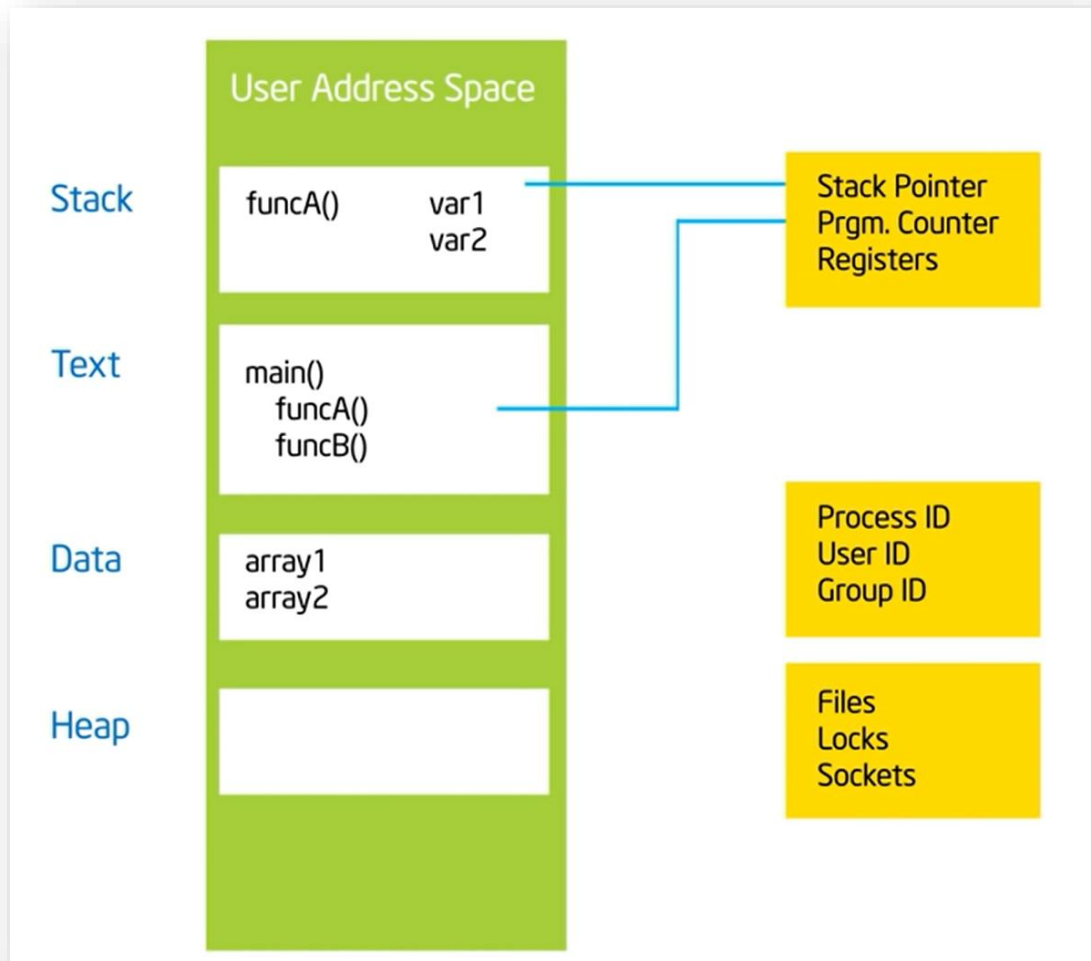
```
if (!rank)
```

```
    recv from all ranks (...)
```

```
else
```

```
    send (0)
```

Processes 0 and 1 – Address Space

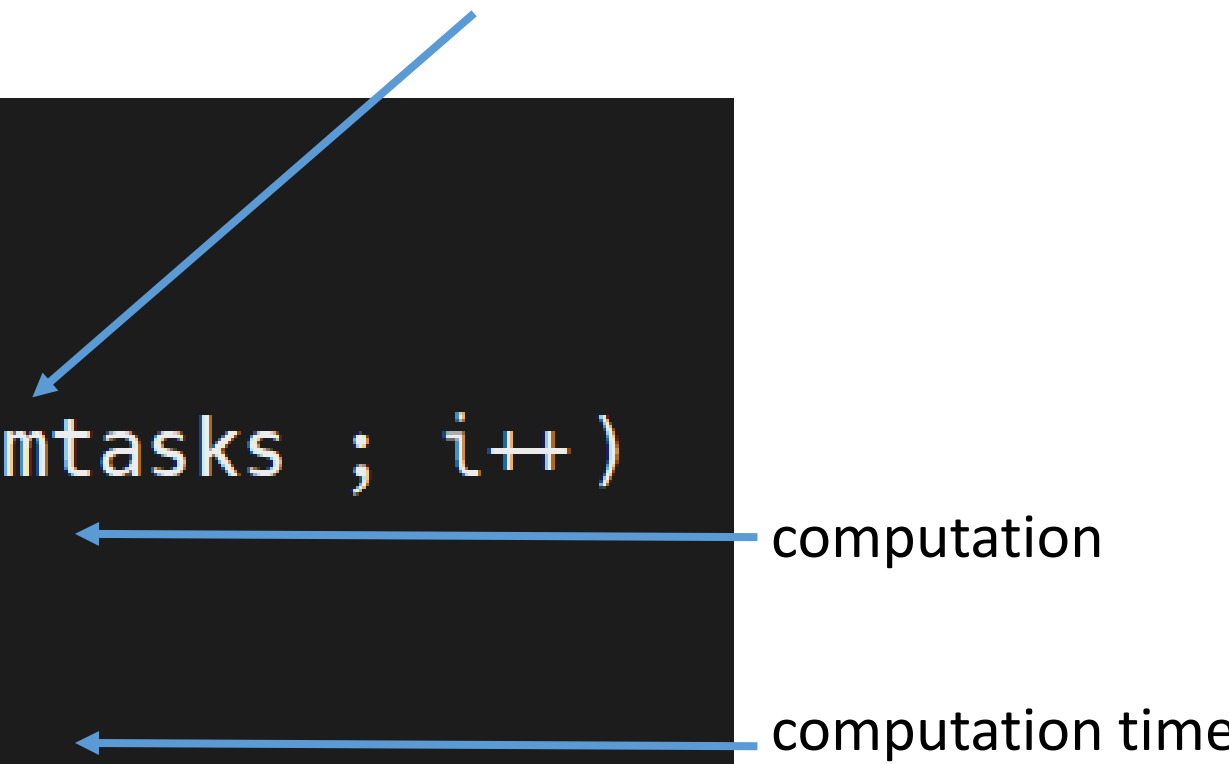


Partial Sum Code – Computation

```
// Initialization  
sidx = rank*N/numtasks;    //start index
```

```
// local sum computation  
sum=0.0;  
stime = MPI_Wtime();  
for (i=sidx; i<sidx+N/numtasks ; i++)  
    sum += a[i] * a[i];  
etime = MPI_Wtime();  
ctime = etime - stime;
```

end index



The diagram consists of three blue arrows pointing from text labels on the right to specific parts of the code. The first arrow points from 'end index' to the expression 'i < sidx + N/numtasks' in the for loop. The second arrow points from 'computation' to the line 'sum += a[i] * a[i];'. The third arrow points from 'computation time' to the line 'ctime = etime - stime;'.

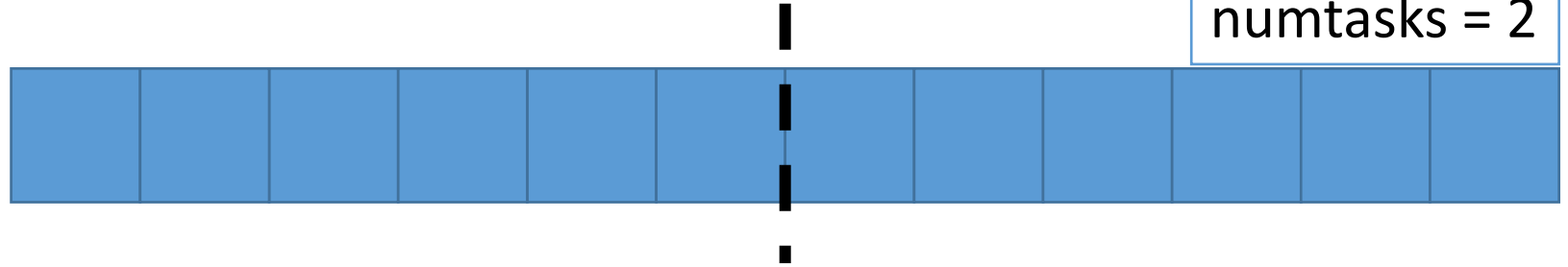
computation

computation time

Parallel Sum Code – Example (2 processes)

```
// Initialization  
sidx = rank*N/numtasks;
```

N = 12
numtasks = 2



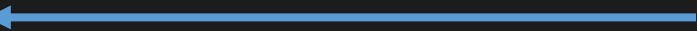
```
// local sum computation  
sum=0.0;  
stime = MPI_Wtime();  
for (i=sidx; i<sidx+N/numtasks ; i++)  
    sum += a[i] * a[i];  
etime = MPI_Wtime();  
ctime = etime - stime;
```

start index
end index

0	1
0	6
5	11

Receive Partial Sums at 0

```
// receive partial sums at rank 0
stime = MPI_Wtime();
if (rank)
{
    MPI_Send(&sum, 1, MPI_DOUBLE, 0, rank, MPI_COMM_WORLD);
}
else
{
    for (int r=1; r<numtasks; r++)
    {
        MPI_Recv(recvarr[r], 1, MPI_DOUBLE, r, r, MPI_COMM_WORLD, &status);
    }
}
etime = MPI_Wtime();
cotime = etime - stime;
```



communication time

```
// local sum computation
sum=0.0;
stime = MPI_Wtime();
for (i=sidx; i<sidx+N/numtasks ; i++)
    sum += a[i] * a[i];
etime = MPI_Wtime();
ctime = etime - stime;
```

ctime -> compute time

```
// receive partial sums at rank 0
stime = MPI_Wtime();
if (rank)
{
    MPI_Send(&sum, 1, MPI_DOUBLE, 0, rank, MPI_COMM_WORLD);
}
else
{
    for (int r=1; r<numtasks; r++)
    {
        MPI_Recv(recvvarr[r], 1, MPI_DOUBLE, r, r, MPI_COMM_WORLD, &status);
    }
}
etime = MPI_Wtime();
cotime = etime - stime;
```

cotime -> communication time

Homework:
Use
MPI_ANY_SOURCE

Timing Results of Partial Sum with Send/Recv Sub-array at Rank 0

Timing

```
class > mpirun -np 2 ./parsum 1000000000
0: Time: 0.121342 0.228165
1: Time: 0.121102 0.228456
class >
class > mpirun -np 4 ./parsum 1000000000
1: Time: 0.062213 0.206533
2: Time: 0.065691 0.236537
0: Time: 0.062425 0.255203
3: Time: 0.062617 0.259298
```


Timing

```
class > mpirun -np 4 ./parsum 1000000000
1: Time: 0.062270 0.198651
2: Time: 0.062419 0.224653
0: Time: 0.063393 0.256728
3: Time: 0.063561 0.256720
class >
class >
class > mpirun -np 4 -hosts csews1,csews21 ./parsum 1000000000
1: Time: 0.060454 2.875476
2: Time: 0.060476 2.917183
3: Time: 0.060519 7.746851
0: Time: 0.060595 7.907365
```

Output

```
mpirun -np 2 ./parsum 10000
```

```
0: Time: 0.000012 0.000068
```

```
1: Time: 0.000011 0.000022
```

```
mpirun -np 2 ./parsum 100000
```

```
0: Time: 0.000113 0.000317
```

```
1: Time: 0.000114 0.000324
```

```
mpirun -np 2 ./parsum 1000000
```

```
0: Time: 0.001250 0.001951
```

```
1: Time: 0.001246 0.001941
```

```
4 processes, 10000 doubles
```

```
0: Time: 0.000006 0.000814
```

```
4 processes, 100000 doubles
```

```
0: Time: 0.000075 0.000764
```

```
4 processes, 1000000 doubles
```

```
0: Time: 0.000662 0.002383
```

Timing – Factors

2 processes, 10000 doubles
0: Time: 0.000012 0.000068

2 processes, 100000 doubles
0: Time: 0.000113 0.000317

2 processes, 1000000 doubles
0: Time: 0.001250 0.001951

4 processes, 10000 doubles
0: Time: 0.000006 0.000814

4 processes, 100000 doubles
0: Time: 0.000075 0.000764

4 processes, 1000000 doubles
0: Time: 0.000662 0.002383

1. Data size impacts communication time
2. Compute time scales to a certain extent
3. Scalability saturation limits change

Timing

Slowest

Compute
time

Communication
time

```
class > mpirun -np 2 ./parsum 10000
1: Time: 0.000012 0.000024
0: Time: 0.000012 0.000024
class >
class >
class >
class > mpirun -np 2 ./parsum 1000000
0: Time: 0.001236 0.002528
1: Time: 0.001242 0.002549
class >
class >
class > mpirun -np 2 ./parsum 1000000000
```

Timing

```
class > mpirun -np 2 ./parsum 10000
0: Time: 0.000012 0.000106
1: Time: 0.000012 0.000023
class >
class > mpirun -np 4 ./parsum 10000
0: Time: 0.000006 0.000038
1: Time: 0.000006 0.000012
2: Time: 0.000007 0.000020
3: Time: 0.000006 0.000018
class >
class > mpirun -np 8 ./parsum 10000
1: Time: 0.000004 0.000023
7: Time: 0.000004 0.000027
3: Time: 0.000004 0.000016
4: Time: 0.000004 0.000012
5: Time: 0.000006 0.000015
2: Time: 0.000006 0.000011
6: Time: 0.000006 0.000015
0: Time: 0.000006 0.000069
```

Over-
decomposition

Communication
time

Timing (One randomly selected rank)

```
class > mpirun -np 2 ./parsum 1000000 | head -1  
0: Time: 0.001225 0.002356  
class >  
class > mpirun -np 4 ./parsum 1000000 | head -1  
1: Time: 0.000746 0.002389  
class >  
class > mpirun -np 8 ./parsum 1000000 | head -1  
1: Time: 0.000534 0.003159  
class >  
class > mpirun -np 12 ./parsum 1000000 | head -1  
1: Time: 0.000556 0.003065  
class >  
class > mpirun -np 16 ./parsum 1000000 | head -1  
1: Time: 0.000263 0.001230
```

Output

Compute
time

```
class $ mpirun -np 18 -f hostfile ./parsum 10000000 | sort -k3n
6: Time: 0.003012 0.060570
7: Time: 0.003012 0.102022
10: Time: 0.003028 0.222636
8: Time: 0.003035 0.144442
9: Time: 0.005718 0.177058
11: Time: 0.005723 0.255620
2: Time: 0.006853 0.012692
3: Time: 0.006865 0.014576
14: Time: 0.007570 0.370506
12: Time: 0.007571 0.287444
17: Time: 0.007571 0.489290
13: Time: 0.007579 0.324584
15: Time: 0.007589 0.408255
16: Time: 0.007607 0.446781
1: Time: 0.008052 0.003379
5: Time: 0.008055 0.010920
0: Time: 0.008059 0.491419
4: Time: 0.008060 0.009140
class $ cat hostfile
csews10:6
csews11:6
csews12:6
csews13:6
csews14:6
csews15:6
csews16:6
csews17:6
csews18:6
class $ █
```

```
class $ mpirun -np 18 -f hostfile ./parsum 10000000 | sort -k4n
2: Time: 0.008899 0.001971
3: Time: 0.008892 0.004043
1: Time: 0.007026 0.004272
4: Time: 0.006996 0.013603
5: Time: 0.007045 0.015271
6: Time: 0.007582 0.043268
7: Time: 0.007561 0.083797
8: Time: 0.007579 0.124421
9: Time: 0.007561 0.162050
10: Time: 0.010233 0.190928
11: Time: 0.010234 0.229928
12: Time: 0.007656 0.276896
13: Time: 0.007631 0.317010
14: Time: 0.010614 0.344311
15: Time: 0.010599 0.386936
16: Time: 0.007566 0.433467
17: Time: 0.007586 0.469213
0: Time: 0.007014 0.480969
class $ cat hostfile
csews10:6
csews11:6
csews12:6
csews13:6
csews14:6
csews15:6
csews16:6
csews17:6
csews18:6
class $ █
```

Communication
time

Compute time

Output

```
6: Time: 0.003012 0.060570
7: Time: 0.003012 0.102022
10: Time: 0.003028 0.222636
8: Time: 0.003035 0.144442
9: Time: 0.005718 0.177058
11: Time: 0.005723 0.255620
2: Time: 0.006853 0.012692
3: Time: 0.006865 0.014576
14: Time: 0.007570 0.370506
12: Time: 0.007571 0.287444
17: Time: 0.007571 0.489290
13: Time: 0.007579 0.324584
15: Time: 0.007589 0.408255
16: Time: 0.007607 0.446781
1: Time: 0.008052 0.003379
5: Time: 0.008055 0.010920
0: Time: 0.008059 0.491419
4: Time: 0.008060 0.009140
```

18 processes

```
csews10:6
csews11:6
csews12:6
csews13:6
csews14:6
csews15:6
csews16:6
csews17:6
csews18:6
```

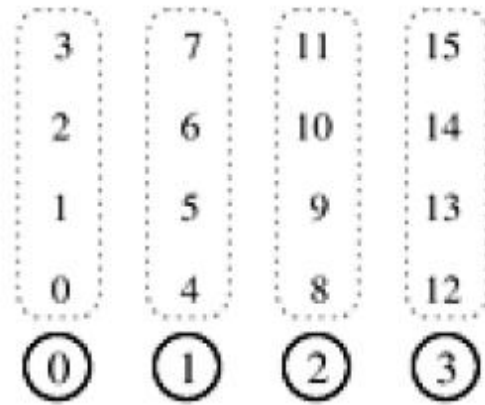
Communication time

```
2: Time: 0.008899 0.001971
3: Time: 0.008892 0.004043
1: Time: 0.007026 0.004272
4: Time: 0.006996 0.013603
5: Time: 0.007045 0.015271
6: Time: 0.007582 0.043268
7: Time: 0.007561 0.083797
8: Time: 0.007579 0.124421
9: Time: 0.007561 0.162050
10: Time: 0.010233 0.190928
11: Time: 0.010234 0.229928
12: Time: 0.007656 0.276896
13: Time: 0.007631 0.317010
14: Time: 0.010614 0.344311
15: Time: 0.010599 0.386936
16: Time: 0.007566 0.433467
17: Time: 0.007586 0.469213
0: Time: 0.007014 0.480969
```

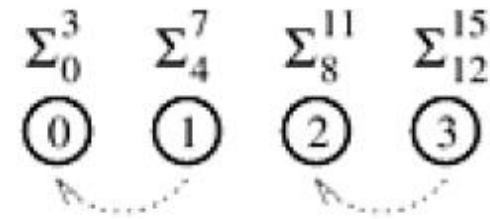

Homework

- Time the partial sum parallel code – report the compute and communication time for sending (1) modified sub-array and (2) partial sums only
 - Scale the data size
 - Scale the process count

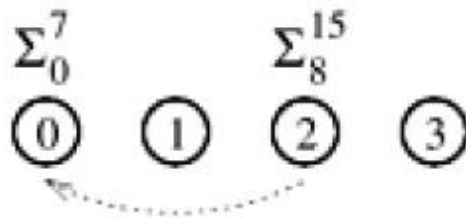
Parallel Sum (Optimized)



(a)



(b)



(c)



(d)

Source: GGKK Chapter 5

Homework

Write the code for optimized parallel sum using MPI_Send and MPI_Recv

Hints:

1. Iterative code -- #steps = ?
2. Whether a rank is sender or receiver or **idle**, depends on the rank and the step number

Code Examples

Send – Recv Example

```
if (myrank == 0) {  
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);  
    MPI_Recv (buf, count, MPI_INT, 1, myrank, MPI_COMM_WORLD, &status);  
}  
else  
if (myrank == 1) {  
    MPI_Recv (buf, count, MPI_INT, 0, myrank, MPI_COMM_WORLD, &status);  
    MPI_Send (buf, count, MPI_INT, 0, 0, MPI_COMM_WORLD);  
}  
  
printf ("%d %d %d\n", myrank, count, status.MPI_SOURCE);
```

mpirun -np 2 ./sendrecv 20

0	20	1
1	20	0

Send – Recv Example

```
if (myrank == 0) {  
    MPI_Recv (buf, count, MPI_INT, 1, myrank, MPI_COMM_WORLD, &status);  
    MPI_Send (buf, count, MPI_INT, 1, 1, MPI_COMM_WORLD);  
}  
else if (myrank == 1) {  
    MPI_Send (buf, count, MPI_INT, 0, 0, MPI_COMM_WORLD);  
    MPI_Recv (buf, count, MPI_INT, 0, myrank, MPI_COMM_WORLD, &status);  
}  
printf ("%d %d %d\n", myrank, count, status.MPI_SOURCE);
```

Find the bug, if any

<https://cse.iitk.ac.in/users/cs633/2025-26-2/bug1.c>

<https://cse.iitk.ac.in/users/cs633/2025-26-2/bug2.c>

Questions?

<https://forms.gle/CuBWQgLrg1YjNNuS9>